

Mathematical Problem Solving II

Advanced Topics and Algorithmic Thinking



Overview

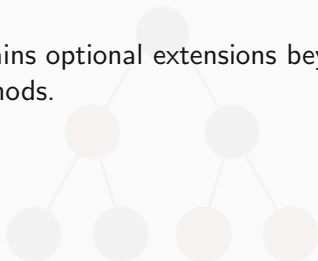
- 1 Advanced Logic And Common Knowledge
 - 2 Advanced Invariants And Monovariants
 - 3 Colouring And Tiling: Generalisations
 - 4 Induction And Recursion: From Proof To Algorithm
 - 5 From Mathematical Insight To Algorithmic Thinking
- 

About This Deck



About This Deck

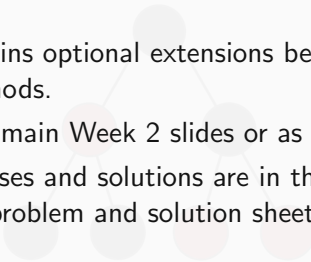
- This deck contains optional extensions beyond the core Week 2 heuristic methods.



About This Deck

- This deck contains optional extensions beyond the core Week 2 heuristic methods.
- Use it after the main Week 2 slides or as follow-up material.

About This Deck

- 
- This deck contains optional extensions beyond the core Week 2 heuristic methods.
 - Use it after the main Week 2 slides or as follow-up material.
 - Matching exercises and solutions are in the advanced sections of the Week 2 problem and solution sheets.

Advanced Logic And Common Knowledge

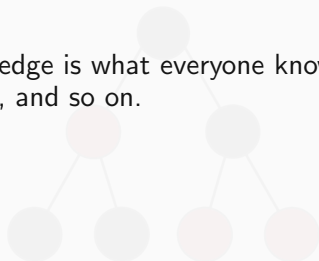


Logic Beyond Two Doors



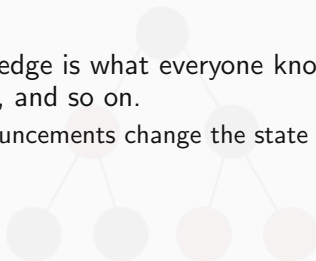
Logic Beyond Two Doors

- Common knowledge is what everyone knows, everyone knows everyone knows, and so on.



Logic Beyond Two Doors

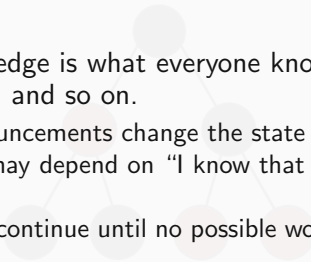
- Common knowledge is what everyone knows, everyone knows everyone knows, and so on.
 - Public announcements change the state of knowledge.



Logic Beyond Two Doors

- Common knowledge is what everyone knows, everyone knows everyone knows, and so on.
 - Public announcements change the state of knowledge.
 - Reasoning may depend on “I know that you know that I know...”

Logic Beyond Two Doors

- 
- Common knowledge is what everyone knows, everyone knows everyone knows, and so on.
 - Public announcements change the state of knowledge.
 - Reasoning may depend on “I know that you know that I know...”
 - Deductions continue until no possible worlds can be eliminated.

Worked Example: Muddy Children (Public Announcement)

n children play outside. k of them have mud on their foreheads.
Each sees the others but not themselves.

Father announces: "At least one of you has mud." Then he repeatedly asks: "Does anyone know their own state?"

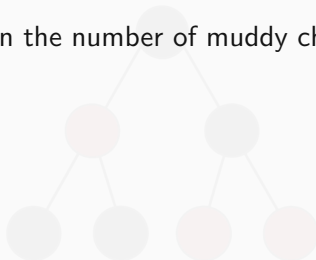
What happens?

Solution: Muddy Children



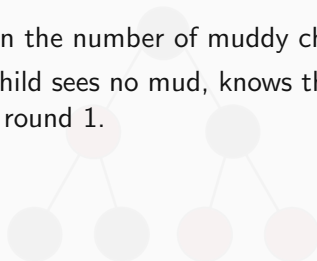
Solution: Muddy Children

→ Use induction on the number of muddy children k .



Solution: Muddy Children

- Use induction on the number of muddy children k .
- If $k = 1$: that child sees no mud, knows they must be the one.
Answers yes on round 1.



Solution: Muddy Children

- Use induction on the number of muddy children k .
- If $k = 1$: that child sees no mud, knows they must be the one. Answers yes on round 1.
- If $k = 2$: each sees one muddy child. If I were clean, the other would see 0 and answer on round 1. They don't. So I must be muddy. Both answer on round 2.

Solution: Muddy Children

- Use induction on the number of muddy children k .
- If $k = 1$: that child sees no mud, knows they must be the one. Answers yes on round 1.
- If $k = 2$: each sees one muddy child. If I were clean, the other would see 0 and answer on round 1. They don't. So I must be muddy. Both answer on round 2.
- Induction: if k , all k answer on round k .

Advanced Invariants And Monovariants

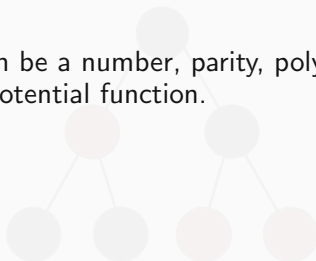


Generalising Invariants



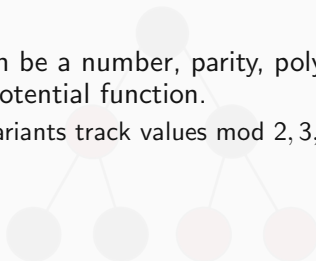
Generalising Invariants

- An invariant can be a number, parity, polynomial, vector modulo m , or potential function.



Generalising Invariants

- An invariant can be a number, parity, polynomial, vector modulo m , or potential function.
 - Modulo invariants track values mod $2, 3, 4, \dots$



Generalising Invariants

- An invariant can be a number, parity, polynomial, vector modulo m , or potential function.
 - Modulo invariants track values mod $2, 3, 4, \dots$
 - Polynomial invariants encode several quantities at once.

Generalising Invariants

- An invariant can be a number, parity, polynomial, vector modulo m , or potential function.
 - Modulo invariants track values mod $2, 3, 4, \dots$
 - Polynomial invariants encode several quantities at once.
 - Potential functions never increase or never decrease.

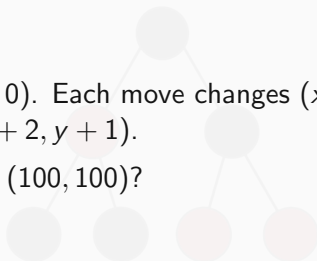
Generalising Invariants

- An invariant can be a number, parity, polynomial, vector modulo m , or potential function.
 - Modulo invariants track values mod $2, 3, 4, \dots$
 - Polynomial invariants encode several quantities at once.
 - Potential functions never increase or never decrease.
 - Several invariants together can prove stronger results.

Worked Example: Grid Jumps

A token starts at $(0,0)$. Each move changes (x,y) to either $(x+1, y+2)$ or $(x+2, y+1)$.

Can the token reach $(100,100)$?



Solution: Grid Jumps



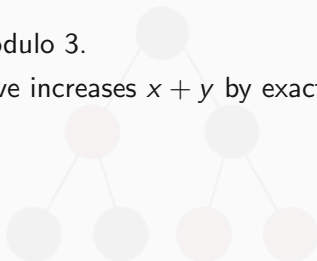
Solution: Grid Jumps

→ Track $x + y$ modulo 3.



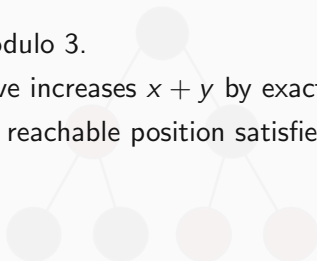
Solution: Grid Jumps

- Track $x + y$ modulo 3.
- Either legal move increases $x + y$ by exactly 3.



Solution: Grid Jumps

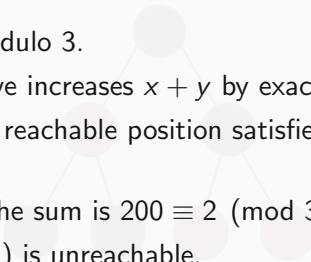
- Track $x + y$ modulo 3.
- Either legal move increases $x + y$ by exactly 3.
- Therefore every reachable position satisfies $x + y \equiv 0 \pmod{3}$.



Solution: Grid Jumps

- Track $x + y$ modulo 3.
- Either legal move increases $x + y$ by exactly 3.
- Therefore every reachable position satisfies $x + y \equiv 0 \pmod{3}$.
- At $(100, 100)$, the sum is $200 \equiv 2 \pmod{3}$.

Solution: Grid Jumps

- 
- Track $x + y$ modulo 3.
 - Either legal move increases $x + y$ by exactly 3.
 - Therefore every reachable position satisfies $x + y \equiv 0 \pmod{3}$.
 - At $(100, 100)$, the sum is $200 \equiv 2 \pmod{3}$.
 - Hence $(100, 100)$ is unreachable.

Colouring And Tiling: Generalisations

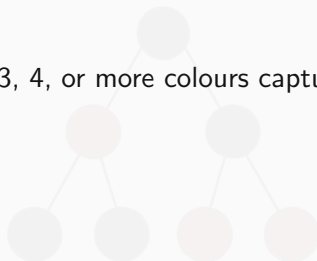


Beyond Two Colours



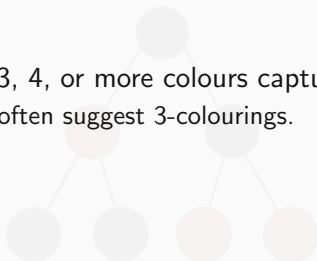
Beyond Two Colours

→ Colouring with 3, 4, or more colours captures more structure.



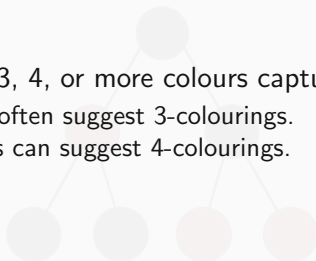
Beyond Two Colours

- Colouring with 3, 4, or more colours captures more structure.
- Triominoes often suggest 3-colourings.

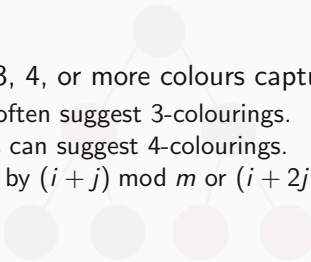


Beyond Two Colours

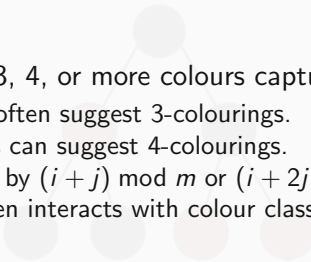
- Colouring with 3, 4, or more colours captures more structure.
 - Triominoes often suggest 3-colourings.
 - Tetrominoes can suggest 4-colourings.



Beyond Two Colours

- 
- Colouring with 3, 4, or more colours captures more structure.
 - Triominoes often suggest 3-colourings.
 - Tetrominoes can suggest 4-colourings.
 - Colour (i, j) by $(i + j) \bmod m$ or $(i + 2j) \bmod m$.

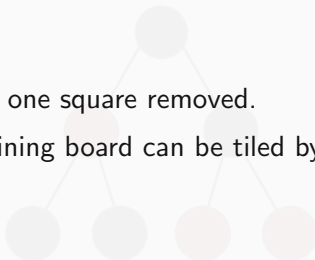
Beyond Two Colours

- 
- Colouring with 3, 4, or more colours captures more structure.
 - Triominoes often suggest 3-colourings.
 - Tetrominoes can suggest 4-colourings.
 - Colour (i, j) by $(i + j) \bmod m$ or $(i + 2j) \bmod m$.
 - Each tile then interacts with colour classes predictably.

Worked Example: Deficient Board

A $2^n \times 2^n$ board has one square removed.

Prove that the remaining board can be tiled by L-shaped trominoes.



Solution: Deficient Board



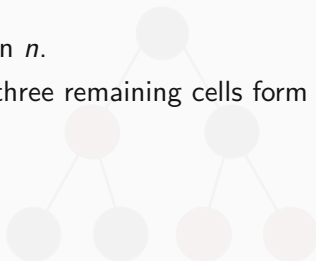
Solution: Deficient Board

→ Use induction on n .



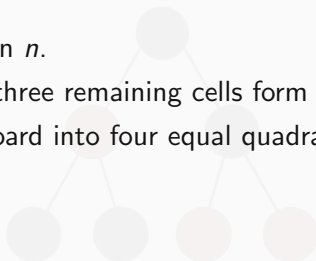
Solution: Deficient Board

- Use induction on n .
- For $n = 1$, the three remaining cells form one tromino.



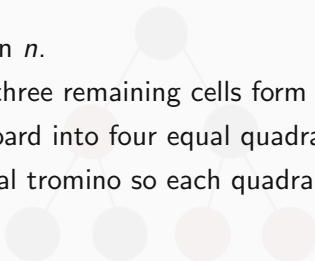
Solution: Deficient Board

- Use induction on n .
- For $n = 1$, the three remaining cells form one tromino.
- Split a larger board into four equal quadrants.

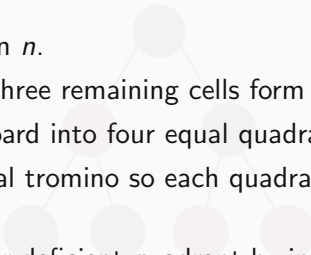


Solution: Deficient Board

- Use induction on n .
- For $n = 1$, the three remaining cells form one tromino.
- Split a larger board into four equal quadrants.
- Place one central tromino so each quadrant has exactly one missing square.



Solution: Deficient Board

- 
- Use induction on n .
 - For $n = 1$, the three remaining cells form one tromino.
 - Split a larger board into four equal quadrants.
 - Place one central tromino so each quadrant has exactly one missing square.
 - Tile each smaller deficient quadrant by induction.

Induction And Recursion: From Proof To Algorithm

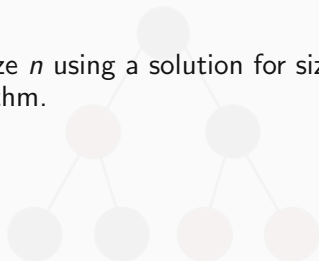


Induction As A Design Principle



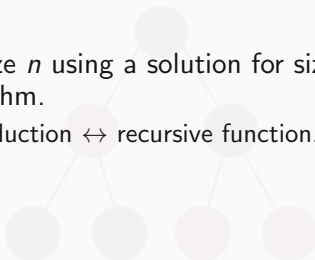
Induction As A Design Principle

→ If I can solve size n using a solution for size $n - 1$, I have a recursive algorithm.

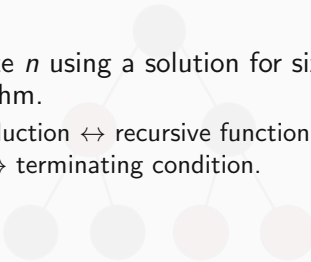


Induction As A Design Principle

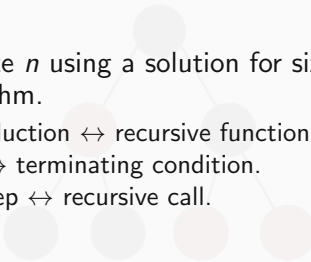
- If I can solve size n using a solution for size $n - 1$, I have a recursive algorithm.
 - Proof by induction $\leftrightarrow$ recursive function.



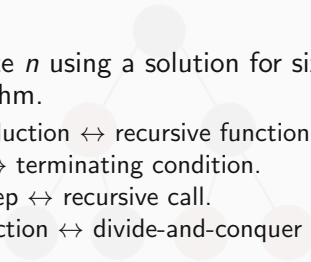
Induction As A Design Principle

- 
- If I can solve size n using a solution for size $n - 1$, I have a recursive algorithm.
 - Proof by induction $\leftrightarrow$ recursive function.
 - Base case $\leftrightarrow$ terminating condition.

Induction As A Design Principle

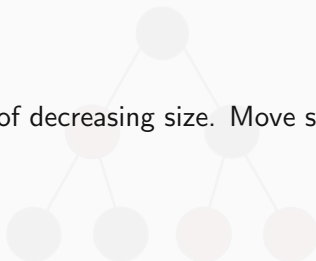
- 
- If I can solve size n using a solution for size $n - 1$, I have a recursive algorithm.
 - Proof by induction $\leftrightarrow$ recursive function.
 - Base case $\leftrightarrow$ terminating condition.
 - Inductive step $\leftrightarrow$ recursive call.

Induction As A Design Principle

- 
- If I can solve size n using a solution for size $n - 1$, I have a recursive algorithm.
 - Proof by induction $\leftrightarrow$ recursive function.
 - Base case $\leftrightarrow$ terminating condition.
 - Inductive step $\leftrightarrow$ recursive call.
 - Strong induction $\leftrightarrow$ divide-and-conquer or DP.

Worked Example: Tower Of Hanoi

Three pegs, n disks of decreasing size. Move stack from peg A to peg C.

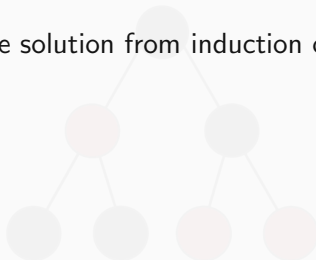


Solution: Tower Of Hanoi



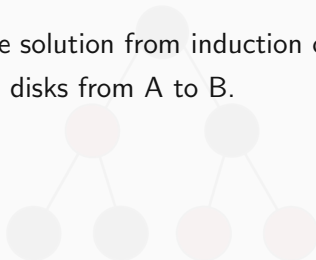
Solution: Tower Of Hanoi

→ Build a recursive solution from induction on n .



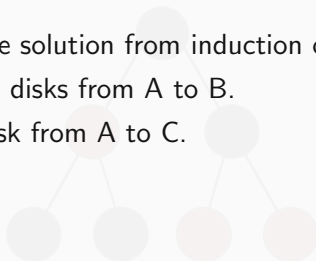
Solution: Tower Of Hanoi

- Build a recursive solution from induction on n .
- Move top $n - 1$ disks from A to B.



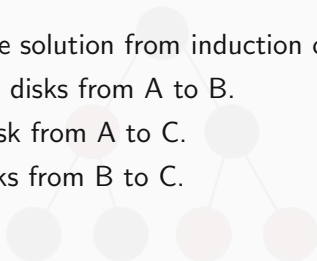
Solution: Tower Of Hanoi

- Build a recursive solution from induction on n .
- Move top $n - 1$ disks from A to B.
- Move largest disk from A to C.

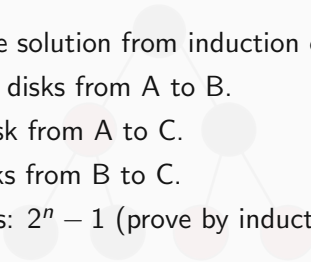


Solution: Tower Of Hanoi

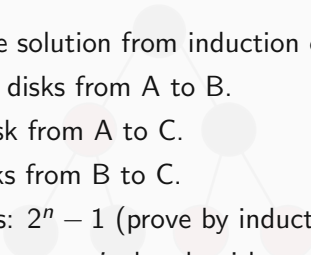
- Build a recursive solution from induction on n .
- Move top $n - 1$ disks from A to B.
- Move largest disk from A to C.
- Move $n - 1$ disks from B to C.



Solution: Tower Of Hanoi

- 
- Build a recursive solution from induction on n .
 - Move top $n - 1$ disks from A to B.
 - Move largest disk from A to C.
 - Move $n - 1$ disks from B to C.
 - Minimum moves: $2^n - 1$ (prove by induction).

Solution: Tower Of Hanoi

- 
- Build a recursive solution from induction on n .
 - Move top $n - 1$ disks from A to B.
 - Move largest disk from A to C.
 - Move $n - 1$ disks from B to C.
 - Minimum moves: $2^n - 1$ (prove by induction).
 - The recursive structure *is* the algorithm.

From Mathematical Insight To Algorithmic Thinking



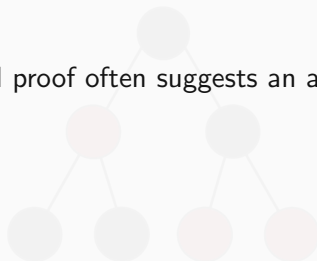
From Mathematical Insight To Algorithmic Thinking

The Bridge To Competitive Programming



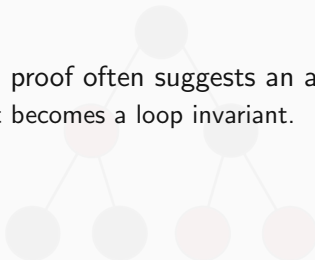
The Bridge To Competitive Programming

→ A mathematical proof often suggests an algorithm directly.



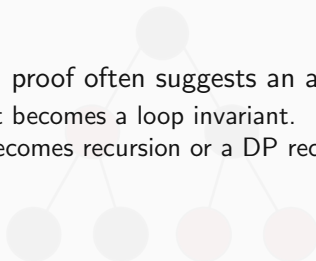
The Bridge To Competitive Programming

- A mathematical proof often suggests an algorithm directly.
- An invariant becomes a loop invariant.

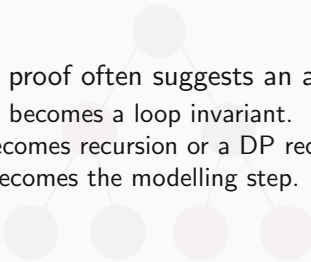


The Bridge To Competitive Programming

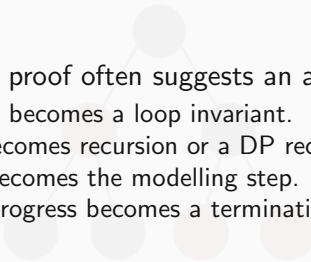
- A mathematical proof often suggests an algorithm directly.
 - An invariant becomes a loop invariant.
 - Induction becomes recursion or a DP recurrence.



The Bridge To Competitive Programming

- 
- A mathematical proof often suggests an algorithm directly.
 - An invariant becomes a loop invariant.
 - Induction becomes recursion or a DP recurrence.
 - Reduction becomes the modelling step.

The Bridge To Competitive Programming

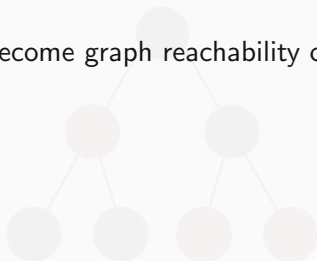
- 
- A mathematical proof often suggests an algorithm directly.
 - An invariant becomes a loop invariant.
 - Induction becomes recursion or a DP recurrence.
 - Reduction becomes the modelling step.
 - A proof of progress becomes a termination argument.

Modelling A Puzzle As A Graph (Algorithmic Bridge)



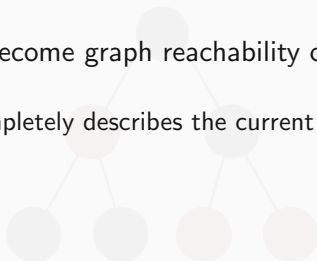
Modelling A Puzzle As A Graph (Algorithmic Bridge)

- Many puzzles become graph reachability or shortest-path problems.



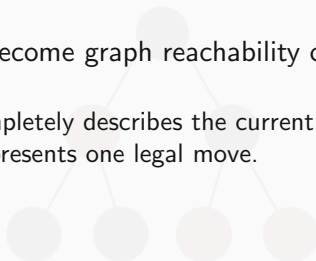
Modelling A Puzzle As A Graph (Algorithmic Bridge)

- Many puzzles become graph reachability or shortest-path problems.
 - A state completely describes the current situation.

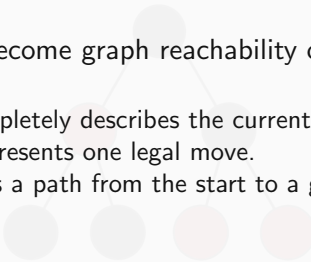


Modelling A Puzzle As A Graph (Algorithmic Bridge)

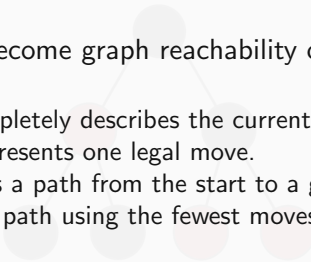
- Many puzzles become graph reachability or shortest-path problems.
 - A state completely describes the current situation.
 - An edge represents one legal move.



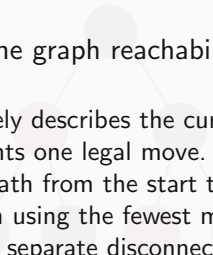
Modelling A Puzzle As A Graph (Algorithmic Bridge)

- 
- Many puzzles become graph reachability or shortest-path problems.
 - A state completely describes the current situation.
 - An edge represents one legal move.
 - A solution is a path from the start to a goal state.

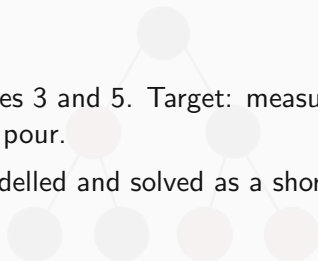
Modelling A Puzzle As A Graph (Algorithmic Bridge)

- 
- Many puzzles become graph reachability or shortest-path problems.
 - A state completely describes the current situation.
 - An edge represents one legal move.
 - A solution is a path from the start to a goal state.
 - BFS finds a path using the fewest moves.

Modelling A Puzzle As A Graph (Algorithmic Bridge)

- 
- Many puzzles become graph reachability or shortest-path problems.
 - A state completely describes the current situation.
 - An edge represents one legal move.
 - A solution is a path from the start to a goal state.
 - BFS finds a path using the fewest moves.
 - An invariant can separate disconnected components.

Worked Example: Water Jugs As A Graph



Two jugs of capacities 3 and 5. Target: measure exactly 4 units.

Allowed: fill, empty, pour.

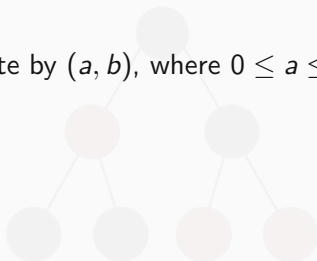
How can this be modelled and solved as a shortest-path problem?

Solution: Water Jugs



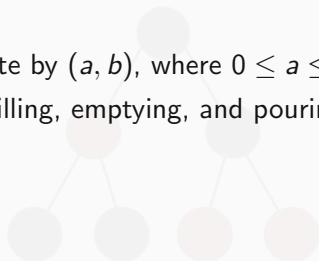
Solution: Water Jugs

→ Represent a state by (a, b) , where $0 \leq a \leq 3$ and $0 \leq b \leq 5$.



Solution: Water Jugs

- Represent a state by (a, b) , where $0 \leq a \leq 3$ and $0 \leq b \leq 5$.
- Add edges for filling, emptying, and pouring between jugs.



Solution: Water Jugs

- Represent a state by (a, b) , where $0 \leq a \leq 3$ and $0 \leq b \leq 5$.
- Add edges for filling, emptying, and pouring between jugs.
- Run BFS from $(0, 0)$ until one coordinate is 4.

Solution: Water Jugs

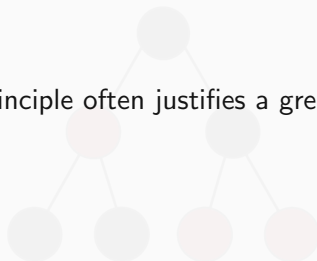
- Represent a state by (a, b) , where $0 \leq a \leq 3$ and $0 \leq b \leq 5$.
- Add edges for filling, emptying, and pouring between jugs.
- Run BFS from $(0, 0)$ until one coordinate is 4.
- One shortest route is
 $(0, 0) \rightarrow (0, 5) \rightarrow (3, 2) \rightarrow (0, 2) \rightarrow (2, 0) \rightarrow (2, 5) \rightarrow (3, 4)$.

Greedy Choices From The Extreme Principle (Algorithmic Bridge)



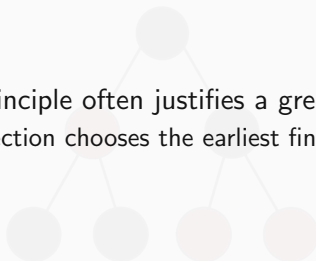
Greedy Choices From The Extreme Principle (Algorithmic Bridge)

→ The extreme principle often justifies a greedy choice.



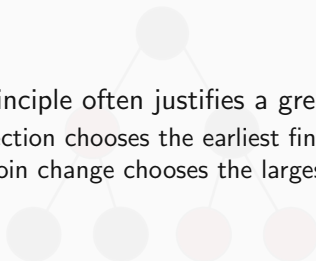
Greedy Choices From The Extreme Principle (Algorithmic Bridge)

- The extreme principle often justifies a greedy choice.
 - Activity selection chooses the earliest finish time.

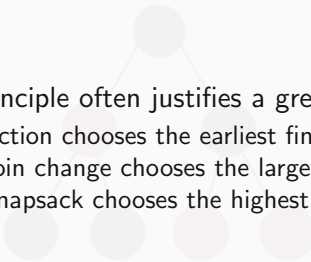


Greedy Choices From The Extreme Principle (Algorithmic Bridge)

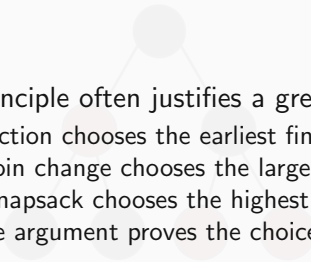
- The extreme principle often justifies a greedy choice.
 - Activity selection chooses the earliest finish time.
 - Canonical coin change chooses the largest coin.



Greedy Choices From The Extreme Principle (Algorithmic Bridge)

- 
- The extreme principle often justifies a greedy choice.
 - Activity selection chooses the earliest finish time.
 - Canonical coin change chooses the largest coin.
 - Fractional knapsack chooses the highest value density.

Greedy Choices From The Extreme Principle (Algorithmic Bridge)

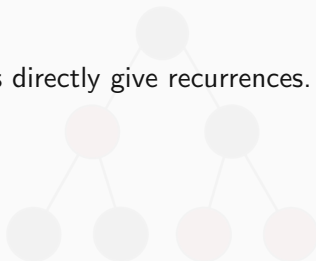
- 
- The extreme principle often justifies a greedy choice.
 - Activity selection chooses the earliest finish time.
 - Canonical coin change chooses the largest coin.
 - Fractional knapsack chooses the highest value density.
 - An exchange argument proves the choice is safe.

Recurrence From Induction (Algorithmic Bridge)



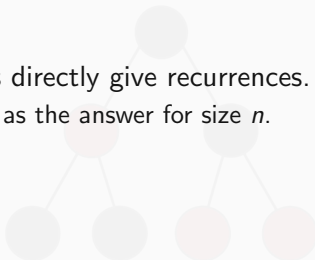
Recurrence From Induction (Algorithmic Bridge)

→ Inductive proofs directly give recurrences.



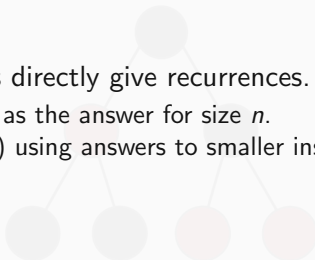
Recurrence From Induction (Algorithmic Bridge)

- Inductive proofs directly give recurrences.
- Define $f(n)$ as the answer for size n .

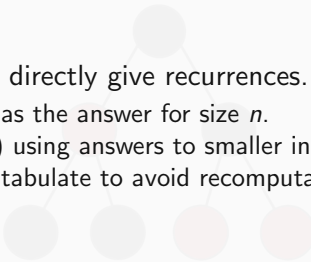


Recurrence From Induction (Algorithmic Bridge)

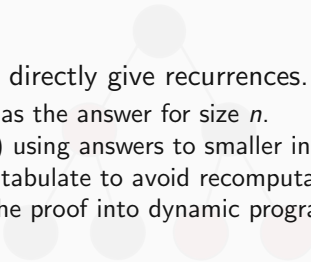
- Inductive proofs directly give recurrences.
 - Define $f(n)$ as the answer for size n .
 - Express $f(n)$ using answers to smaller instances.



Recurrence From Induction (Algorithmic Bridge)

- 
- Inductive proofs directly give recurrences.
 - Define $f(n)$ as the answer for size n .
 - Express $f(n)$ using answers to smaller instances.
 - Memoise or tabulate to avoid recomputation.

Recurrence From Induction (Algorithmic Bridge)

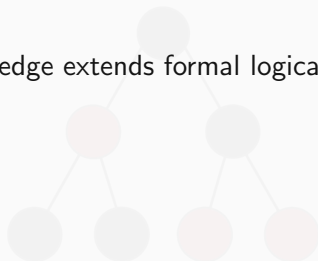
- 
- Inductive proofs directly give recurrences.
 - Define $f(n)$ as the answer for size n .
 - Express $f(n)$ using answers to smaller instances.
 - Memoise or tabulate to avoid recomputation.
 - This turns the proof into dynamic programming.

Wrap-Up



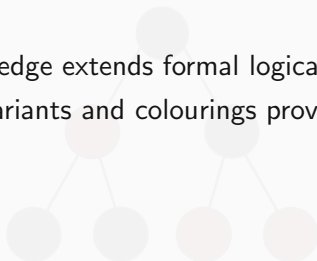
Wrap-Up

→ Common knowledge extends formal logical reasoning.



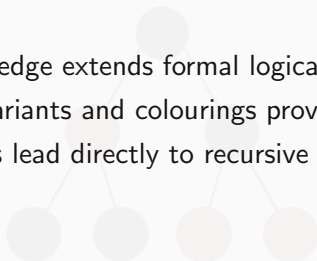
Wrap-Up

- Common knowledge extends formal logical reasoning.
- Generalised invariants and colourings prove stronger results.

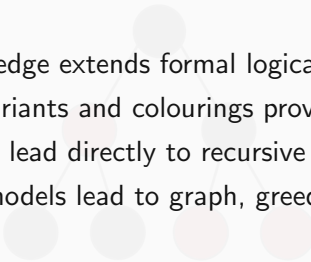


Wrap-Up

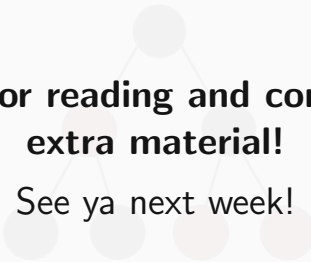
- Common knowledge extends formal logical reasoning.
- Generalised invariants and colourings prove stronger results.
- Inductive proofs lead directly to recursive algorithms.



Wrap-Up

- 
- Common knowledge extends formal logical reasoning.
 - Generalised invariants and colourings prove stronger results.
 - Inductive proofs lead directly to recursive algorithms.
 - Mathematical models lead to graph, greedy, and DP techniques.

Thanks for going further!



**Thank you for reading and completing the
extra material!**

See ya next week!